XTrace Specification v1.0

AI-Native Debug Trace Format for SystemVerilog

Release: 2026-05-05

1. Introduction

XTrace is a debug trace format intended to replace or complement VCD when the consumer is not only a waveform viewer, but also a parser, debug database, formal or debug analysis tool, AI agent, or LLM-based root-cause assistant.

XTrace is designed to be compact, structured, semantic, streamable, easy to parse, easy to summarize, and better aligned with hardware transactions than raw signal toggles.

This revision strengthens the specification in five areas:

1. determinism and replay semantics

2. text-format escaping and compatibility rules

3. memory and bulk-data transport

4. block framing, integrity, and indexability

5. extension and forward-compatibility behavior

2. Design Goals

Compared to VCD, XTrace aims to provide:

1. dictionary-based naming to avoid repeating long hierarchy strings

2. delta-oriented signal dump so only changes are emitted

3. transaction-first modeling for requests, responses, instructions, bursts, exceptions, and cache activity

4. AI-friendly semantic records for assertions, events, causality, and context bundles

5. multi-level observability across waveform-level, transaction-level, and summary-level views

6. text and binary forms for easy bring-up and efficient deployment

7. deterministic replay semantics so tools reconstruct the same state from the same stream

8. explicit extension points so vendors can add features without breaking baseline parsers

3. Format Variants

3.1 XTrace-T

Human-readable text format. Recommended for bring-up, debugging the parser or emitter, diffing traces, and simple tool interoperability.

3.2 XTrace-B

Compact binary format. Recommended for long regressions, large SoCs, high-frequency traces, database ingest, and cloud processing.

This specification focuses mainly on XTrace-T while also defining the binary packet model.

4. Conceptual Model

XTrace consists of five layers:

4.1 Dictionary layer

Static metadata including modules, signals, enums, transaction schemas, tags, source maps, and optional cycle domains.

4.2 Signal delta layer

Time progression plus changed signal values.

4.3 Semantic event layer

Higher-level records such as instruction decode and retire, FSM transitions, assertion pass and fail, and reset or power transitions.

4.4 Transaction layer

Transaction lifecycle records for begin, update, end, abort, linkage, and summaries.

4.5 Framing and indexing layer

Optional blocks, checksums, summaries, and retrieval boundaries for large traces.

5. File Structure

An XTrace text file is composed of top-level directives followed by sections.

Example:

```
@xtrace 1.0
@format text
@timescale 1ns
@design cpu16
@profile architectural

@section dict
...
@section trace
...
@section transactions
...
```

```
@section end
```

6. Top-Level Directives

6.1 Version directive

```
@xtrace 1.0
```

Required. Declares the file format version.

6.2 Format directive

```
@format text
```

Allowed values: text, binary.

6.3 Timescale directive

```
@timescale 1ns
```

Defines the unit of timestamps and deltas.

6.4 Design directive

```
@design cpu16
```

Optional design name.

6.5 Profile directive

```
@profile architectural
```

Optional. Indicates intended debug scope. Recommended values: minimal, architectural, bus, microarch, full_debug, xezim_ai_debug.

6.6 Producer directive

```
@producer <tool_name> <tool_version>
```

Optional. Identifies the trace producer.

6.7 Capabilities directive

```
@capabilities <capability_list>
```

Optional. Declares optional record families or producer features.

6.8 Compression directive

`@compression <none|zstd>`

Optional. Indicates transport compression applied to the file or blocks.

6.9 Extension policy directive

`@extensions <ignore_unknown|fail_unknown|vendor_allowed>`

Optional. Declares expected behavior when unknown record types or attributes are encountered.

7. Sections

The following sections are defined:

- `@section dict`
- `@section trace`
- `@section transactions`
- `@section end`

Optional sections:

- `@section summaries`
- `@section memory`
- `@section strings`
- `@section vendor.<name>`

Rules:

- Section order shall be dict first, then zero or more trace-like sections, then end.
- A consumer shall ignore unknown optional sections unless @extensions fail_unknown is declared.
- A producer should not emit records outside a declared section.

8. General Record and Text Rules

8.1 Character encoding

XTrace-T shall use UTF-8.

8.2 Line model

Each record occupies one logical line. A producer shall not split one record across multiple physical lines.

8.3 Comments

Lines beginning with # are comments and may be ignored by consumers.

8.4 Whitespace

Leading and trailing ASCII whitespace around a record line may be ignored. Whitespace inside tokens is significant unless quoted.

8.5 Quoted strings

A quoted string uses double quotes. Supported escapes are:

- `\\` for literal backslash
- `\"` for literal quote
- `\n` for newline
- `\t` for horizontal tab
- `\xHH` for a byte value

8.6 Key ordering

Within a record, key-value pairs may appear in any order unless the record syntax fixes positional fields before attributes.

8.7 Unknown attributes

Consumers shall preserve unknown attributes when round-tripping records if practical. Otherwise they shall ignore them unless @extensions fail_unknown is declared.

8.8 Determinism

Given the same ordered stream of valid XTrace records and the same initial empty consumer state, compliant consumers shall reconstruct the same visible signal and transaction state.

9. Dictionary Records

The dictionary section defines all static entities used by later trace records.

9.1 Module record

Syntax:

```
M,<module_id>,<hier_path>
```

9.2 Signal record

Syntax:

```
S,<signal_id>,<module_id>,<name>,<type>[,tags=<tag1>|<tag2>|...][,enc=<mode>]
[,alias=<canonical_signal_id>][,width=<bits>][,reset=<value>]
```

9.3 Signal types

Recommended scalar and vector types:

- `bit`
- `u8, u16, u32, u64`
- `s8, s16, s32, s64`
- `logic[N]`
- `enum:<enum_name>`
- `str`
- `bytes`
- `mem[addr_width,data_width]`

9.4 Enum record

Syntax:

```
E,<enum_name>,<value0>=<name0>,<value1>=<name1>,...
```

9.5 Transaction schema record

Syntax:

```
TQ,<txn_type_id>,<txn_type_name>,fields=<f1>|<f2>|...[,tags=<t1>|<t2>|...][,required=<r1>|
<r2>|...]
```

9.6 Clock domain record

Syntax:

```
CD,<clock_id>,signal=<signal_id>,edge=<posedge|negedge|both>[,domain=<name>]
```

9.7 File/source record

Syntax:

```
F,<file_id>,<path>[,sha=<sha256>]
```

9.8 Source-location record

Syntax:

```
SL,<object_id>,file=<file_id>,line=<line>[,col=<col>][,decl=<name>]
```

10. Trace Records

10.1 Time record

Syntax:

T,+<delta>[,d=<delta_index>][,r=<region>]

10.2 Single signal delta record

Syntax:

```
D,<signal_id>,<value>
```

10.3 Packed signal delta record

Syntax:

```
P,<signal_id>=<value>[,<signal_id>=<value>]*
```

10.4 Event record

Syntax:

```
X,<event_type>[,<key>=<value>]*
```

10.5 Assertion record

Syntax:

```
A,<kind>[,<key>=<value>]*
```

10.6 Snapshot record

Syntax:

```
N,<snap_type>[,<signal_id>=<value>]*
```

10.7 Comment record

Syntax:

C,"free text comment"

10.8 Cycle marker

Syntax:

```
CY,<clock_id>,+<cycle_delta>[,edge=<edge>]
```

10.9 Block markers

Syntax:

```
IB,B,id=<block_id>[,<key>=<value>]*
```

```
IB,E,id=<block_id>[,<key>=<value>]*
```

11. Transaction Records

11.1 Transaction lifecycle model

Every transaction may progress through begin, zero or more updates, and end or abort. Transactions may also reference a parent transaction, reference child transactions, bind to relevant signals, and bind to assertions.

11.2 Transaction begin record

Syntax:

```
Q,B,id=<txn_id>,type=<txn_type>[,<key>=<value>]*
```

11.3 Transaction update record

Syntax:

```
Q,U,id=<txn_id>[,<key>=<value>]*
```

11.4 Transaction end record

Syntax:

```
Q,E,id=<txn_id>,status=<status>[,<key>=<value>]*
```

11.5 Transaction abort record

Syntax:

```
Q,X,id=<txn_id>,reason=<reason>[,<key>=<value>]*
```

11.6 Transaction link record

Syntax:

```
Q,L,parent=<parent_id>,child=<child_id>,relation=<relation>
```

11.7 Transaction summary record

Syntax:

QS,<key>=<value>[,<key>=<value>]*

12. Memory and Bulk Data Records

12.1 Purpose

The base signal dump should not be forced to encode large memory changes as many scalar signal deltas. This section adds dedicated memory-oriented records.

12.2 Memory write record

Syntax:

```
MW,<mem_id>,addr=<addr>,data=<value>[,be=<mask>][,parent=<txn_id>]
```

12.3 Memory read observation record

Syntax:

```
MR,<mem_id>,addr=<addr>,data=<value>[,parent=<txn_id>]
```

12.4 Memory block record

Syntax:

```
MB,<mem_id>,base=<addr>,count=<count>,data=<encoded_blob>[,encoding=<hex|base64|
zstd_base64>]
```

13. Standard Transaction Types

Recommended canonical transaction names include:

- inst
- branch
- load
- store
- interrupt
- exception
- mem_read
- mem_write
- axi_read
- axi_write
- apb_read
- apb_write
- dma_read
- dma_write
- cache_lookup
- cache_fill
- cache_evict
- writeback
- snoop
- invalidate
- clean
- reset_seq

- power_up
- power_down
- retention_save
- retention_restore

14. Recommended Tags

Recommended tags include:

- clock
- reset
- pc
- instruction
- fsm
- state
- register
- architectural
- bus
- req_addr
- req_data
- rsp_data
- control
- handshake_valid
- handshake_ready
- assertion_context
- memory
- axi
- apb
- interrupt
- power

15. Value Representation

15.1 Numeric values

Use hex for vectors unless decimal is semantically better.

15.2 Enums

Use symbolic names where possible.

15.3 Unknowns

Allowed representations include X, Z, 0xXX0A, and 4s:bits=1001,xmask=0010,zmask=0100.

15.4 Strings

Strings should be quoted if containing spaces or punctuation.

15.5 Opaque blobs

Bytes and bulk-data fields may use base64 or producer-declared encodings.

16. Correlation Rules

16.1 Assertion to transaction

Use parent=<txn_id> in assertion records.

16.2 Transaction to signals

Use ctx= in transaction begin or update records.

16.3 Instruction to sub-transaction

Use parent=i44 in bus or memory transactions.

16.4 Source mapping

Use file and line attributes or SL dictionary records whenever a parser or AI agent will present source-backed diagnoses.

17. Core Example

```
@xtrace 1.0
@format text
@timescale 1ns
@design cpu16
@profile architectural
@producer xezim 0.1.0
@capabilities signal_delta|transactions|assertions|source_map|sv_region|context_bundle

@section dict
M,m0,/top
M,m1,/top/cpu
S,s0,m1,pc,u16,tags=pc|state,enc=delta
S,s1,m1,ir,u16,tags=instruction,enc=abs
S,s2,m1,state,enum:cpu_state,tags=fsm|state,enc=enum
S,s3,m1,mem_addr,u16,tags=req_addr|bus
S,s4,m1,mem_rdata,u16,tags=rsp_data|bus
S,s5,m1,mem_wr,bit,tags=control|bus
S,s6,m1,r1,u16,tags=register|architectural
F,f0,rtl/cpu16.sv,sha=8d41...
SL,s0,file=f0,line=44,decl=pc_q
E,cpu_state,0=RESET,1=FETCH,2=DECODE,3=EXEC,4=MEM,5=WB
TQ,tt0,inst,fields=pc|opcode|mnemonic|rd|result|status|latency,tags=instruction
TQ,tt1,mem_read,fields=addr|size|rdata|status|latency|parent,tags=bus|memory

@section trace
T,+0
N,full,s0=0x0000,s2=RESET,s5=0,s6=0x0000

T,+1
P,s2=FETCH,s0=0x0000
X,inst_fetch,pc=0x0000
```

```
T,+1
P,s1=0x9123,s2=DECODE
X,inst_decode,name=LOAD,pc=0x0000,rd=r1,addr=0x0123

T,+1
P,s2=MEM,s3=0x0123,s5=0
MW,mem0,addr=0x0123,data=0x00AA,parent=t100

T,+1
P,s4=0x00AA

T,+1
P,s2=WB,s6=0x00AA

@section transactions
Q,B,id=i44,type=tt0,pc=0x0000,opcode=0x9123,mnemonic=LOAD,rd=r1,ctx=s0|s1|s2|s6
Q,U,id=i44,stage=decode
Q,B,id=t100,type=tt1,addr=0x0123,size=2,parent=i44,agent=cpu0,ctx=s3|s4|s5
Q,E,id=t100,status=ok,rdata=0x00AA,latency=1
Q,U,id=i44,stage=wb
Q,E,id=i44,status=retired,result=0x00AA,latency=3

@section end
```

18. Text Grammar (BNF)

file := header section+

header := version_decl format_decl? timescale_decl? design_decl? profile_decl? producer_decl? capabilities_decl? compression_decl? extensions_decl?

version_decl := "@xtrace" version

format_decl   := "@format" ("text" | "binary")

timescale_decl := "@timescale" token

design_decl   := "@design" token

profile_decl   := "@profile" token

producer_decl   := "@producer" token token

capabilities_decl := "@capabilities" token

compression_decl := "@compression" ("none" | "zstd")

extensions_decl := "@extensions" ("ignore_unknown" | "fail_unknown" | "vendor_allowed")

section   := dict_section | trace_section | txn_section | memory_section | vendor_section | end_section

dict_section   := "@section dict" newline dict_record+

trace_section   := "@section trace" newline trace_record+

txn_section   := "@section transactions" newline txn_record+

memory_section := "@section memory" newline memory_record+

vendor_section := "@section vendor." name newline vendor_record+

end_section   := "@section end"

dict_record   := module_rec | signal_rec | enum_rec | txn_schema_rec | clock_domain_rec | file_rec | source_loc_rec

module_rec   := "M," id "," path

signal_rec   := "S," id "," id "," name "," type attrs*

enum_rec   := "E," name "," enum_item ("," enum_item)*

txn_schema_rec := "TQ," id "," name "," attrs*

clock_domain_rec:= "CD," id "," kvpair ("," kvpair)*

file_rec   := "F," id "," path ("," kvpair)*

source_loc_rec := "SL," id "," kvpair ("," kvpair)*

trace_record   := time_rec | delta_rec | packed_rec | event_rec | assert_rec | snap_rec | comment_rec | cycle_rec | block_rec | memory_record

time_rec   := "T,+" integer ("," kvpair)*

```
delta_rec       := "D," id "," value

packed_rec      := "P," assign ("," assign)*

event_rec       := "X," name ("," kvpair)*

assert_rec      := "A," name ("," kvpair)*

snap_rec        := "N," name ("," assign)*

comment_rec     := "C," quoted_string

cycle_rec       := "CY," id ",+" integer ("," kvpair)*

block_rec       := "IB," ("B" | "E") "," kvpair ("," kvpair)*


txn_record      := txn_begin | txn_update | txn_end | txn_abort | txn_link | txn_summary

txn_begin       := "Q,B," kvpair ("," kvpair)*

txn_update      := "Q,U," kvpair ("," kvpair)*

txn_end         := "Q,E," kvpair ("," kvpair)*

txn_abort       := "Q,X," kvpair ("," kvpair)*

txn_link        := "Q,L," kvpair ("," kvpair)*

txn_summary     := "QS," kvpair ("," kvpair)*


memory_record   := mem_write | mem_read | mem_block

mem_write       := "MW," id "," kvpair ("," kvpair)*

mem_read        := "MR," id "," kvpair ("," kvpair)*

mem_block       := "MB," id "," kvpair ("," kvpair)*
```

19. Semantic Model

19.1 General semantics

XTrace is a simulator or tool evidence stream, not merely a rendered waveform dump. Records are interpreted in sequence. The current simulation context consists of the active time, and optionally additional profile-defined context such as cycle domain or scheduler region.

19.2 Dictionary semantics

Dictionary records define stable identities and metadata. Once introduced, an ID retains its meaning throughout the file.

19.3 Time semantics

A T record advances or rebinds the current time context. All subsequent D, P, X, A, N, MW, MR, MB, and Q records inherit the most recent time until another T record appears.

19.4 Signal semantics

A D or P record changes the current value of one or more signals in the reconstructed state model. Signal values persist until explicitly changed.

19.5 Event semantics

An X record annotates semantically meaningful activity but does not itself require a persistent state update unless a consumer chooses to derive one.

19.6 Assertion semantics

An A record represents assertion, checker, or property outcomes. An A,fail record is intended to be directly consumable by a parser or AI agent and may carry context signals, parent linkage, source mapping, and severity.

19.7 Snapshot semantics

An N record materializes a compact state image. N,full is intended as a checkpoint. N,ctx is intended as a local evidence packet around a failure, event, or transaction boundary.

19.8 Transaction semantics

A transaction ID is created by Q,B. It remains live until Q,E or Q,X. Q,U refines or extends the transaction state. Parent-child links form a causality graph across instructions, bus activity, memory requests, cache activity, and failures.

19.9 Memory semantics

```
MW, MR, and MB describe memory-observable activity rather than ordinary scalar net updates.
Consumers may reconstruct partial or full memory state from these records if requested.
```

19.10 Compatibility semantics

Unknown record families in known sections shall be ignored unless strict failure is requested through @extensions fail_unknown. Unknown attributes shall not invalidate otherwise well-formed records.

20. Binary Model

Recommended packet classes:

- 0x01 module dictionary
- 0x02 signal dictionary
- 0x03 enum dictionary
- 0x04 transaction schema dictionary
- 0x05 clock domain dictionary
- 0x06 file dictionary
- 0x07 source-location dictionary
- 0x10 time delta
- 0x11 signal delta
- 0x12 packed signal delta block
- 0x13 cycle marker
- 0x14 block marker
- 0x20 event
- 0x21 assertion
- 0x22 snapshot
- 0x23 memory write
- 0x24 memory read
- 0x25 memory block
- 0x40 transaction begin
- 0x41 transaction update
- 0x42 transaction end
- 0x43 transaction abort
- 0x44 transaction link
- 0x45 transaction summary

Recommended primitive encodings:

```
- ULEB128 for IDs and time deltas
- zigzag varint for signed values
- string table for repeated strings
- optional zstd block compression
- optional per-block checksum for integrity
```

21. Compression Recommendations

To keep the format compact:

1. use dictionary IDs for repeated names

2. use time deltas rather than absolute times

3. use packed signal records when multiple signals change

4. prefer transaction and memory records over dumping every intermediate net

5. emit periodic snapshots only when useful

6. dump architectural or semantic signals by default

7. keep microarchitectural signals profile-gated

8. use enum coding for FSM states

9. allow XOR or arithmetic delta encoding for counters and PC

10. compress the binary stream with zstd

11. chunk long runs into index blocks for retrieval-oriented readers

22. Emitter Model

Recommended emission sources:

```
- simulator core
- DPI-C callback
- UVM monitor
- generated instrumentation module
```

```
- custom debug interface in a homegrown simulator
```

Recommended sampling points:

```
- positive clock edge
- bus handshake completion
- instruction retire
- assertion trigger
- reset or power transitions
- memory interface beats
- explicit monitor boundaries
```

Emitter flow:

1. initialize dictionary

2. assign stable signal IDs

3. track previous values

4. emit T,+delta

5. emit P,... for changed watched signals

6. emit Q,... for transactions

7. emit MW/MR/MB for memory-visible activity when enabled

8. emit A,... for assertions

9. emit N,ctx,... on important failures

10. emit IB,B / IB,E around retrieval or summary chunks when useful

23. Parser-Facing Normalized Model

A parser should expose at least these internal objects:

```
- signal dictionary
- current signal state
- live transaction table
```

```
- optional memory model
- source map
- block index
```

24. Compliance Levels

Level 0: minimal

```
- dictionary
- time
- signal deltas
```

Level 1: semantic

```
- level 0 plus events and assertions
```

Level 2: transactional

```
- level 1 plus transaction lifecycle and parent-child linkage
```

Level 3: AI-native

```
- level 2 plus signal tags, context snapshots, summaries, and transaction schemas
```

Level 4: retrieval-oriented

```
- level 3 plus index blocks, source mapping, and optional memory records
```

Recommended target: Level 3 or Level 4 depending on workload.

Appendix A. Xezim Native Profile

A.1 Purpose

The Xezim Native Profile defines how Xezim should generate XTrace records directly from simulation. It keeps the core specification generic while allowing Xezim to provide a richer AI-debug stream.

The profile name is:

```
@profile xezim_ai_debug
```

The producer declaration is:

```
@producer xezim <version>
```

Example header:

```
@xtrace 1.0

@format text

@producer xezim 0.1.0

@timescale 1ps

@design openc910_subsystem

@profile xezim_ai_debug

@capabilities signal_delta|transactions|assertions|source_map|sv_region|context_bundle|
index_blocks
```

Implementation modes:

```
- raw_delta

- semantic

- ai_debug
```

Recommended Xezim default: ai_debug.

Appendix B. Final Definition

XTrace v1.0 is a dictionary-based, delta-encoded, semantic and transactional hardware debug trace format designed for efficient parser implementation and direct AI-agent reasoning. It additionally defines compatibility, framing, and bulk-data mechanisms suitable for large simulator-native debug streams.